HEAP EXPLOITATION

M1 CyberSchool 2026

AMOSSYS

Organisation

- 3 * 1h30 de CM
  - Les slides sont volontairement peu détaillées
  - Vous avez de la place pour prendre des notes dessus
- 6 * 1h30 de TP
  - Mise en place d'un environnement d'exploitation
  - Plusieurs séries de challenges
- Evaluation
  - Les TP seront notés
  - Des questions de cours seront posées dans l'évaluation finale
- Objectif
  - Maîtriser les bases d'un debugger (gdb)
  - Comprendre en détail le fonctionnement de la heap
  - Identifier rapidement les mauvaises pratiques dans du code

Sommaire

P1 : Explications

- Qu'est-ce que la HEAP
- Les problèmes qu'on y trouve
- Certaines protections

P2 : Exploitation

- TCache Poisonning
- Overlapping1
- malloc hook
- libc leak
- Overlapping2
- Consolidation

P3 : Tricks

- ret2plt_ret2main
- one_gadget
- shellcoding

Ressources

- Pwncollege (vidéos et challenges)
  - https://pwn.college/program-security/dynamic-allocator-misuse/
  - https://pwn.college/software-exploitation/dynamic-allocator-exploitation/
- How2Heap
  - Tous types de vulns détaillées en fonctions des versions de libc
  - https://github.com/shellphish/how2heap

PARTIE 1

Explications

AMOSSYS

Qu'est-ce que la HEAP

Les différentes sections d'un binaire

- Sections les plus importantes
  - .text
  - .rodata
  - .bss
  - .data
- Résolution dynamique
  - .plt
  - .got
  - .plt.got
  - .got.plt

```
(gdb) info file
0x0000000000000350 - 0x0000000000000370 is .note.gnu.property
0x0000000000000370 - 0x0000000000000394 is .note.gnu.build-id
0x0000000000000394 - 0x00000000000003b0 is .interp
0x00000000000003b0 - 0x00000000000003d4 is .gnu.hash
0x00000000000003d8 - 0x0000000000000480 is .dynsym
0x0000000000000480 - 0x000000000000050f is .dynstr
0x0000000000000510 - 0x000000000000051e is .gnu.version
0x0000000000000520 - 0x0000000000000550 is .gnu.version_r
0x0000000000000550 - 0x0000000000000610 is .rela.dyn
0x0000000000000610 - 0x0000000000000628 is .rela.plt
0x0000000000001000 - 0x0000000000001017 is .init
0x0000000000001020 - 0x0000000000001040 is .plt
0x0000000000001040 - 0x0000000000001048 is .plt.got
0x0000000000001050 - 0x0000000000001179 is .text
0x000000000000117c - 0x0000000000001185 is .fini
0x0000000000002000 - 0x0000000000002004 is .rodata
0x0000000000002004 - 0x0000000000002030 is .eh_frame_hdr
0x0000000000002030 - 0x00000000000020dc is .eh_frame
0x00000000000020dc - 0x00000000000020fc is .note.ABI-tag
0x0000000000003dd0 - 0x0000000000003dd8 is .init_array
0x0000000000003dd8 - 0x0000000000003de0 is .fini_array
0x0000000000003de0 - 0x0000000000003fc0 is .dynamic
0x0000000000003fc0 - 0x0000000000003fe8 is .got
0x0000000000003fe8 - 0x0000000000004008 is .got.plt
0x0000000000004008 - 0x0000000000004018 is .data
0x0000000000004018 - 0x0000000000004020 is .bss
```

Qu'est-ce que la HEAP

AMOSSYS

Les différentes sections d'un binaire

- Sections les plus importantes
  - .text
  - .rodata
  - .bss
  - .data
- Résolution dynamique
  - .plt
  - .got
  - .plt.got
  - .got.plt

```
(gdb) info file
0x0000000000000350 - 0x0000000000000370 is .note.gnu.property
0x0000000000000370 - 0x0000000000000394 is .note.gnu.build-id
0x0000000000000394 - 0x00000000000003b0 is .interp
0x00000000000003b0 - 0x00000000000003d4 is .gnu.hash
0x00000000000003d8 - 0x0000000000000480 is .dynsym
0x0000000000000480 - 0x000000000000050f is .dynstr
0x0000000000000510 - 0x000000000000051e is .gnu.version
0x0000000000000520 - 0x0000000000000550 is .gnu.version_r
0x0000000000000550 - 0x0000000000000610 is .rela.dyn
0x0000000000000610 - 0x0000000000000628 is .rela.plt
0x0000000000001000 - 0x0000000000001017 is .init
0x0000000000001020 - 0x0000000000001040 is .plt
0x0000000000001040 - 0x0000000000001048 is .plt.got
0x0000000000001050 - 0x0000000000001179 is .text
0x000000000000117c - 0x0000000000001185 is .fini
0x0000000000002000 - 0x0000000000002004 is .rodata
0x0000000000002004 - 0x0000000000002030 is .eh_frame_hdr
0x0000000000002030 - 0x00000000000020dc is .eh_frame
0x00000000000020dc - 0x00000000000020fc is .note.ABI-tag
0x0000000000003dd0 - 0x0000000000003dd8 is .init_array
0x0000000000003dd8 - 0x0000000000003de0 is .fini_array
0x0000000000003de0 - 0x0000000000003fc0 is .dynamic
0x0000000000003fc0 - 0x0000000000003fe8 is .got
0x0000000000003fe8 - 0x0000000000004008 is .got.plt
0x0000000000004008 - 0x0000000000004018 is .data
0x0000000000004018 - 0x0000000000004020 is .bss
```

Qu'est-ce que la HEAP

AMOSSYS

Où stocker

On voudrait stocker des données

- Entre les fonctions
- De manière 'permanente'

Où ça ?

Qu'est-ce que la HEAP

AMOSSYS

Où stocker

On voudrait stocker des données

- Entre les fonctions
- De manière 'permanente'

Où ça ?

- Dans .plt/.got/.plt.got/.got.plt ?
- Dans .text ?
- Dans .rodata ?
- Dans .data ?
- Dans .bss ?

Qu'est-ce que la HEAP

AMOSSYS

Où stocker

On voudrait stocker des données

- Entre les fonctions
- De manière 'permanente'

Où ça ?

- Dans .plt/.got/.plt.got/.got.plt ? Non
  - c'est déjà utilisé
  - c'est entouré par d'autres données donc inextensible
- Dans .text ?
- Dans .rodata ?
- Dans .data ?
- Dans .bss ?

Qu'est-ce que la HEAP

AMOSSYS

Où stocker

On voudrait stocker des données

- Entre les fonctions
- De manière 'permanente'

Où ça ?

- Dans .plt/.got/.plt.got/.got.plt ?
- Dans .text ? Non
  - c'est en lecture seule
  - entouré et inextensible aussi
- Dans .rodata ?
- Dans .data ?
- Dans .bss ?

Qu'est-ce que la HEAP

AMOSSYS

Où stocker

On voudrait stocker des données

- Entre les fonctions
- De manière 'permanente'

Où ça ?

- Dans .plt/.got/.plt.got/.got.plt ?
- Dans .text ?
- Dans .rodata ? Non
  - c'est en lecture seule
  - entouré et inextensible aussi
- Dans .data ?
- Dans .bss ?

Qu'est-ce que la HEAP

AMOSSYS

Où stocker

On voudrait stocker des données

- Entre les fonctions
- De manière 'permanente'

Où ça ?

- Dans .plt/.got/.plt.got/.got.plt ?
- Dans .text ?
- Dans .rodata ?
- Dans .data ? Quasiment
  - c'est déjà déclaré et de taille fixe
  - il n'y a que .bss derrière
  - et c'est read-write
- Dans .bss ?

Qu'est-ce que la HEAP

AMOSSYS

Où stocker

On voudrait stocker des données

- Entre les fonctions
- De manière 'permanente'

Où ça ?

- Dans .plt/.got/.plt.got/.got.plt ?
- Dans .text ?
- Dans .rodata ?
- Dans .data ?
- Dans .bss ? Oui
  - déjà déclaré
  - mais il n'y a rien derrière
  - et c'est read-write

Qu'est-ce que la HEAP

AMOSSYS

brk

```
man brk

NAME
       brk, sbrk - change data segment size

LIBRARY
       Standard C library (libc, -lc)

SYNOPSIS
       #include <unistd.h>

       int brk(void *addr);
       void *sbrk(intptr_t increment);

syscall : brk ; rax : 12(0xC) ;        rdi : unsigned long brk
```

Qu'est-ce que la HEAP

brk

```c
#include <unistd.h>
#include <stdio.h>
#include <stdlib.h>


int main(){
    void * new_seg_end = sbrk(0x200);
    return 0;
}
```

```
 $ strace ./brk_sbrk3
execve("./brk_sbrk3", ...) = 0
...
brk(NULL)        = 0x55f361bd0000
brk(0x55f361bd0200)    = 0x55f361bd0200
...
```

Qu'est-ce que la HEAP

brk

```
(gdb) disas main
Dump of assembler code for function main:
0x0000000000001139 <+0>:     push   %rbp
0x000000000000113a <+1>:     mov    %rsp,%rbp
0x000000000000113d <+4>:     sub    $0x10,%rsp
0x0000000000001141 <+8>:     mov    $0x200,%edi
0x0000000000001146 <+13>:    call   0x1030 <sbrk@plt>
0x000000000000114b <+18>:    mov    %rax,-0x8(%rbp)
0x000000000000114f <+22>:    mov    $0x0,%eax
0x0000000000001154 <+27>:    leave
0x0000000000001155 <+28>:    ret
```

Qu'est-ce que la HEAP

brk

```
(gdb) b *main+13
Breakpoint 1 at 0x1146
(gdb) b *main+18
Breakpoint 2 at 0x114b
Breakpoint 1, 0x0000555555555146 in main ()
(gdb) info proc map
0x0000555555558000 0x0000555555559000 0x1000    0x3000    rw-p  /.../brk_sbrk3
0x00007ffff7d9e000 0x00007ffff7da1000 0x3000    0x0       rw-p
Breakpoint 2, 0x000055555555514b in main ()
(gdb) info proc map
0x0000555555558000 0x0000555555559000 0x1000    0x3000    rw-p  /.../brk_sbrk3
0x0000555555559000 0x000055555555a000 0x1000    0x0       rw-p  [heap]
0x00007ffff7d9e000 0x00007ffff7da1000 0x3000    0x0       rw-p
```

Qu'est-ce que la HEAP

la heap

Une nouvelle 'section' apparait lors de l'exécution

```
(gdb) info proc map
Start Addr         End Addr           Size               Offset             Perms File
0x0000555555554000 0x0000555555555000 0x1000             0x0                r--p  /.../heap_example
0x0000555555555000 0x0000555555556000 0x1000             0x1000             r-xp  /.../heap_example
0x0000555555556000 0x0000555555557000 0x1000             0x2000             r--p  /.../heap_example
0x0000555555557000 0x0000555555558000 0x1000             0x2000             r--p  /.../heap_example
0x0000555555558000 0x0000555555559000 0x1000             0x3000             rw-p  /.../heap_example
0x0000555555559000 0x000055555557a000 0x21000            0x0                rw-p  [heap]
0x00007ffff7dab000 0x00007ffff7dae000 0x3000             0x0                rw-p
------------------ ------------------ ------             ---                ----  libc
0x00007ffff7f97000 0x00007ffff7fa4000 0xd000             0x0                rw-p
0x00007ffff7fbf000 0x00007ffff7fc1000 0x2000             0x0                rw-p
0x00007ffff7fc1000 0x00007ffff7fc5000 0x4000             0x0                r--p  [vvar]
0x00007ffff7fc5000 0x00007ffff7fc7000 0x2000             0x0                r-xp  [vdso]
------------------ ------------------ ------             ---                ----  linker
0x00007ffff7ffe000 0x00007ffff7fff000 0x1000             0x0                rw-p
0x00007ffffffde000 0x00007ffffffff000 0x21000            0x0                rw-p  [stack]
```

Qu'est-ce que la HEAP

la heap

- Heureusement nous n'allons pas developper en utilisant brk,
- La libc met à disposition un premier wrapper (qui reste très bas niveau) : 'sbrk()'.
- Elle met aussi à disposition de nombreux wrappers et algorithmes pour manipuler cette nouvelle 'section'.
- Allocation
  - malloc
  - calloc
  - realloc
  - ...
- Liberation
  - free
  - realloc
  - ...

Qu'est-ce que la HEAP

Checkpoint 1

Ce que vous devez avoir retenu ici :

- La heap est un nouvel endroit mappé au runtime.
- Obtenu par extension du segment data (RW) contenant .data/.bss avec l'appel système 'brk'.
- La libc met à disposition les fonctions 'malloc' et 'free' qui sont des abstractions haut niveau pour manipuler cette nouvelle 'section'.

Qu'est-ce que la HEAP

malloc/free/chunk

Avec ce petit example on va voir ce que font

- malloc, sur stack et sur heap
- free, sur stack et sur heap
- et on va introduire le concept de chunk

```
int main(int argc, char ** argv){
    void * result_malloc = malloc(16);

    strcpy(result_malloc, "aaaa");
    printf("0x%lx\n", result_malloc);
    printf("%s\n", result_malloc);

    free(result_malloc);
    printf("0x%lx\n", result_malloc);
    printf("%s\n", result_malloc);
}
```

Qu'est-ce que la HEAP

malloc

```c
int main(int argc, char ** argv){
    void * result_malloc = malloc(16);

    strcpy(result_malloc, "aaaa");
    printf("0x%lx\n", result_malloc);
    printf("%s\n", result_malloc);
    //break ici
    free(result_malloc);
    printf("0x%lx\n", result_malloc);
    printf("%s\n", result_malloc);
}
```

```
printf("0x%lx\n", result_malloc); -> 0x5555555592a0
printf("%s\n", result_malloc);    -> aaaa

(gdb) x/10gx $rsp
0x7fffffffda60: 0x007fffffffdb98 0x000001f7fe4780
0x7fffffffda70: 0x00000000000000 [0x005555555592a0]
0x7fffffffda80: 0x00000000000001 0x007ffff7dcaca8
0x7fffffffda90: 0x007fffffffdb80 0x00555555555169
0x7fffffffdaa0: 0x00000155554040 0x007fffffffdb98

(gdb) x/4gx 0x00005555555592a0-16
0x555555559290: 0x00000000000000 0x00000000000021
0x5555555592a0: [0x00000061616161] 0x00000000000000
```

Qu'est-ce que la HEAP

free

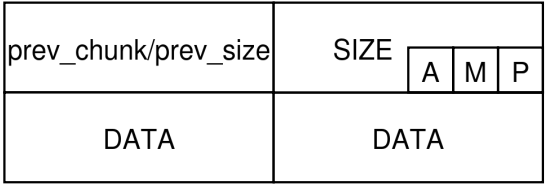
```c
int main(int argc, char ** argv){
    void * result_malloc = malloc(16);

    strcpy(result_malloc, "aaaa");
    printf("0x%lx\n", result_malloc);
    printf("%s\n", result_malloc);
    free(result_malloc);
    printf("0x%lx\n", result_malloc);
    printf("%s\n", result_malloc);
    //break ici
}
```

```
printf("0x%lx\n", result_malloc); -> 0x5555555592a0
printf("%s\n", result_malloc);    -> YUUU

(gdb) x/10gx $rsp
0x7fffffffda60: 0x007fffffffdb98 0x000001f7fe4780
0x7fffffffda70: 0x00000000000000 [0x005555555592a0]
0x7fffffffda80: 0x00000000000001 0x007ffff7dcaca8
0x7fffffffda90: 0x007fffffffdb80 0x00555555555169
0x7fffffffdaa0: 0x00000155554040 0x007fffffffdb98

(gdb) x/10gx 0x00005555555592a0-16
0x555555559290: 0x00000000000000 0x00000000000021
0x5555555592a0: [0x00000000000000] 0x000000000000
```

Qu'est-ce que la HEAP

chunk

```
malloc :
(gdb) x/4gx 0x00005555555592a0-16
0x555555559290: 0x00000000000000 0x0000000021
0x5555555592a0: [0x00000061616161] 0x00000000
```



- malloc retourne un pointeur qui pointe vers data
- mais sur heap c'est toute une structure qui est allouée
- structure appelée chunk
- les 8 premiers octets sont relatifs au chunk d'avant(passons)
- les 8 octets d'après contiennent la taille du chunk (incluant tout le chunk)
- et des flags :
  - A -> Le chunk appartient à la main arena ?
  - M -> Le chunk a-t-il été obtenu via mmap ?
  - P -> Est-ce que le chunk précédent est utilisé ?
- le reste est pour le contenu du chunk

Qu'est-ce que la HEAP

chunk

Quelques subtilités sur la taille

```
#include <unistd.h>
#include <stdio.h>
#include <stdlib.h>

int main(int argc, char ** argv){
  void * a = malloc(24);
  void * b = malloc(16);
  void * c = malloc(25);
  void * d = malloc(0);
  void * e = malloc(7);
}
```

→ quelle sera la taille réelle de ces chunks ?

Qu'est-ce que la HEAP

chunk

Quelques subtilités sur la taille

```c
#include <unistd.h>
#include <stdio.h>
#include <stdlib.h>

int main(int argc, char ** argv){
  void * a = malloc(24);
  void * b = malloc(16);
  void * c = malloc(25);
  void * d = malloc(0);
  void * e = malloc(7);
}
```

```
0x555555559290: 0x0000000000000 0x0000000000021
0x5555555592a0: 0x0000000000000 0x0000000000000
0x5555555592b0: 0x0000000000000 0x0000000000021
0x5555555592c0: 0x0000000000000 0x0000000000000
0x5555555592d0: 0x0000000000000 0x0000000000031
0x5555555592e0: 0x0000000000000 0x0000000000000
0x5555555592f0: 0x0000000000000 0x0000000000000
0x555555559300: 0x0000000000000 0x0000000000021
0x555555559310: 0x0000000000000 0x0000000000000
0x555555559320: 0x0000000000000 0x0000000000021
0x555555559330: 0x0000000000000 0x0000000000000
0x555555559340: 0x0000000000000 0x0000000020cc1
0x555555559350: 0x0000000000000 0x0000000000000
```

Qu'est-ce que la HEAP

chunk

Quelques subtilités sur la taille

```c
#include <unistd.h>
#include <stdio.h>
#include <stdlib.h>

int main(int argc, char ** argv){
  void * a = malloc(24);
  void * b = malloc(16);
  void * c = malloc(25);
  void * d = malloc(0);
  void * e = malloc(7);
}
```

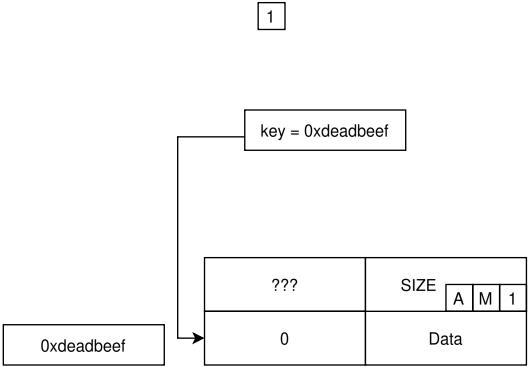
```
0x555555559290: 0x0000000000000 0x0000000000021
0x5555555592a0: 0x0000000000000 0x0000000000000
0x5555555592b0: 0x0000000000000 0x0000000000021
0x5555555592c0: 0x0000000000000 0x0000000000000
0x5555555592d0: 0x0000000000000 0x0000000000031
0x5555555592e0: 0x0000000000000 0x0000000000000
0x5555555592f0: 0x0000000000000 0x0000000000000
0x555555559300: 0x0000000000000 0x0000000000021
0x555555559310: 0x0000000000000 0x0000000000000
0x555555559320: 0x0000000000000 0x0000000000021
0x555555559330: 0x0000000000000 0x0000000000000
0x555555559340: 0x0000000000000 0x0000000020cc1
0x555555559350: 0x0000000000000 0x0000000000000
```

→ A quoi correspond cette ligne ?

Qu'est-ce que la HEAP

chunk

```
free :
(gdb) x/10gx 0x00005555555592a0-16
0x555555559290: 0x00000000000 0x00000000021
0x5555555592a0: [0x00000000000] 0x000000000
```



Qu'est-ce que la HEAP

chunk

Quelques subtilités sur le free

```
#include <unistd.h>
#include <stdio.h>
#include <stdlib.h>

int main(int argc, char ** argv){
  void * a = malloc(24);
  free(a);   (1)
  void * b = NULL;
  free(b);   (2)
  void * c = &main;
  free(c);   (3)
  free(a+0x100);   (4)
  unsigned long test[4] = {0, 0x21, 0, 0};
  void * chunk = &test[2];
  free(chunk);   (5)
}
```

→ Lesquels vont marcher, et lesquels vont échouer ?

Qu'est-ce que la HEAP

chunk

Quelques subtilités sur le free

```
#include <unistd.h>
#include <stdio.h>
#include <stdlib.h>

int main(int argc, char ** argv){
  void * a = malloc(24);
  free(a);   (1)
  void * b = NULL;
  free(b);   (2)
  void * c = &main;
  free(c);   (3)
  free(a+0x100);   (4)
  unsigned long test[4] = {0, 0x21, 0, 0};
  void * chunk = &test[2];
  free(chunk);   (5)
}
```

- 1) Cas classique, ça marche.
- 2) Ça marche aussi de free(0), ça ne fait juste rien.
- 3) Marche pas, peut être car non inscriptible ?
- 4) Marche pas non plus pourtant c'est inscriptible.
- 5) Marche à partir du moment ou c'est inscriptible et que ça ressemble à un chunk.

Qu'est-ce que la HEAP

previnuse

```c
int main(int argc, char ** argv){
    void * a = malloc(2048);
    void * b = malloc(2048);
    strcpy(a, "aaaaaaaaaaaaaaaa");
    strcpy(b, "bbbbbbbbbbbbbbbb");
    // break 1
    free(a);
    // break 2
    return 0;
}
```

```
break 1 :
0x555555559eb0: 0x0000000000000000 0x0000000000000811
0x555555559ec0: 0x6262626262626262 0x0062626262626262
0x555555559ed0: 0x0000000000000000 0x0000000000000000

break 2 :
0x555555559eb0: 0x0000000000000810 0x0000000000000810
0x555555559ec0: 0x6262626262626262 0x0062626262626262
0x555555559ed0: 0x0000000000000000 0x0000000000000000
```

Qu'est-ce que la HEAP

previnuse

- Vous remarquerez que j'ai utilisé un chunk de taille 2048 et pas 16 comme d'habitude
- C'est parce que certains mécanismes d'optimisation sont mis en place par la libc
- Les chunks qui sont free sont stockés dans des listes pour des soucis de rapidité
- Ainsi quand un chunk nouvellement demandé peut être pris depuis une de ces listes, cela évite de refaire une allocation
- il existe plusieurs types de listes, tcache, fastbin, unsortedbin, largebin et smallbin.
- Nous allons nous concentrer sur le tcache.

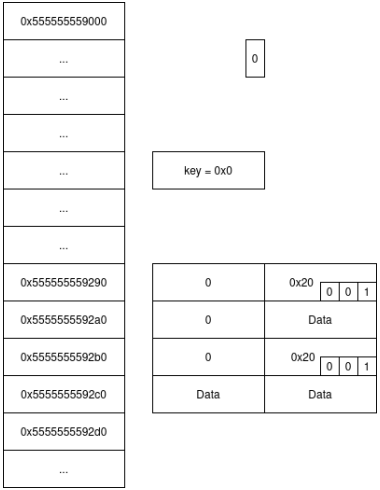
Qu'est-ce que la HEAP

tcache

- Un ensemble de listes
- Listes simplement chainées
- de taille 0 à 1032(oui on peut malloc(0))
- 7 chunks maximum par liste
- Première structure que les algos de Heap (malloc/free) vont regarder
- (le previnuse byte n'est pas utilisé dans le tcache, d'où l'exemple précedent avec une taille 2048)

Qu'est-ce que la HEAP

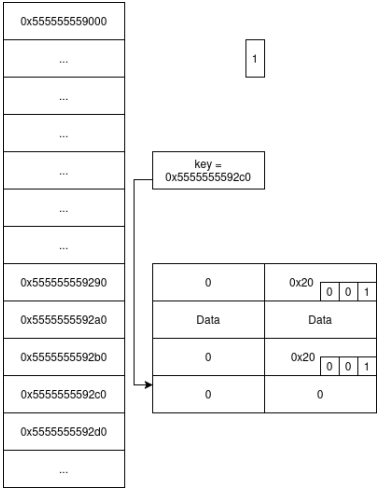
tcache




```c
#include <unistd.h>
#include <stdio.h>
#include <stdlib.h>

int main(int argc, char ** argv){
    void * c1 = malloc(16);
    void * c2 = malloc(16);
    // break ici
    free(c2);
    free(c1);
    c1 = malloc(16);
    c2 = malloc(16);
}
```

Qu'est-ce que la HEAP

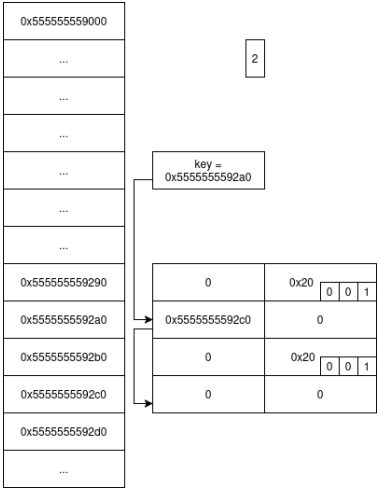
tcache




```c
#include <unistd.h>
#include <stdio.h>
#include <stdlib.h>

int main(int argc, char ** argv){
    void * c1 = malloc(16);
    void * c2 = malloc(16);
    free(c2);
    // break ici
    free(c1);
    c1 = malloc(16);
    c2 = malloc(16);
}
```

Qu'est-ce que la HEAP

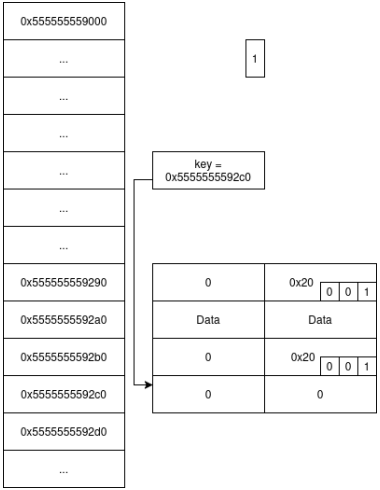
tcache




```c
#include <unistd.h>
#include <stdio.h>
#include <stdlib.h>

int main(int argc, char ** argv){
    void * c1 = malloc(16);
    void * c2 = malloc(16);
    free(c2);
    free(c1);
    // break ici
    c1 = malloc(16);
    c2 = malloc(16);
}
```

Qu'est-ce que la HEAP

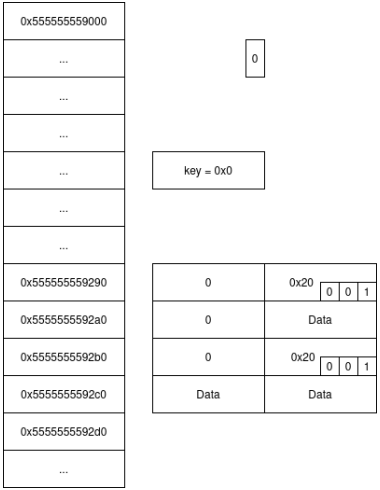
tcache




```c
#include <unistd.h>
#include <stdio.h>
#include <stdlib.h>

int main(int argc, char ** argv){
    void * c1 = malloc(16);
    void * c2 = malloc(16);
    free(c2);
    free(c1);
    c1 = malloc(16);
    // break ici
    c2 = malloc(16);
}
```

Qu'est-ce que la HEAP

tcache




```c
#include <unistd.h>
#include <stdio.h>
#include <stdlib.h>

int main(int argc, char ** argv){
    void * c1 = malloc(16);
    void * c2 = malloc(16);
    free(c2);
    free(c1);
    c1 = malloc(16);
    c2 = malloc(16);
    // break ici
}
```

Qu'est-ce que la HEAP

tcache

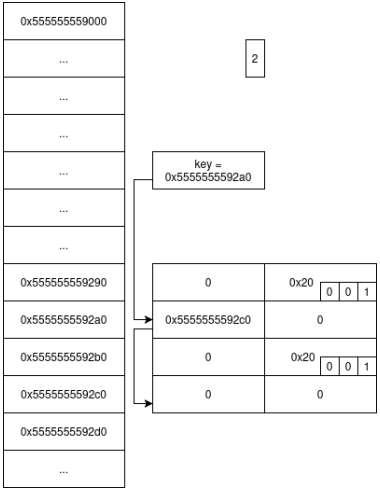
- Tout le tcache (toutes les listes qu'il contient) est handle par une seule structure
- cette structure est placée en tout début de heap et est créée lors de la première allocation.

```
#define TCACHE_MAX_BINS 64

typedef struct tcache_entry {
    struct tcache_entry *next;
    struct tcache_perthread_struct *key;
} tcache_entry;

typedef struct tcache_perthread_struct
{
  char counts[TCACHE_MAX_BINS];
  tcache_entry *entries[TCACHE_MAX_BINS];
} tcache_perthread_struct;
```

- 64 bytes qui servent à compter le nombre de chunk dans chaque liste
- 64 pointeurs qui servent à identifier le premier élément de chaque liste.

Qu'est-ce que la HEAP

tcache



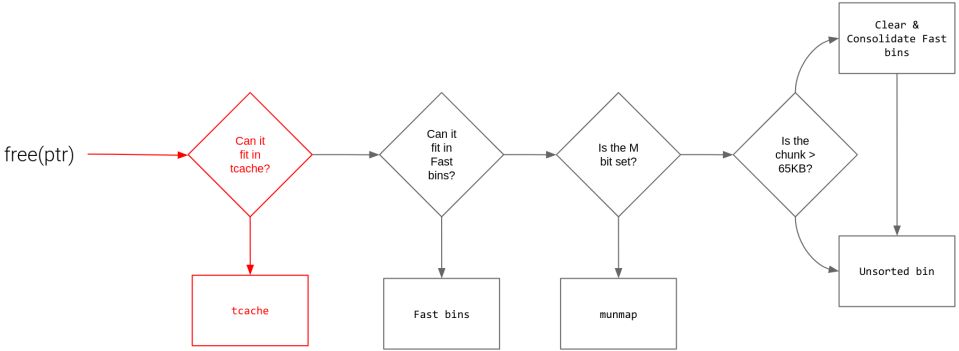

```
0x555555559000: 0x000000000000 0x000000000291
0x555555559010: 0x000000000002 0x000000000000
0x555555559020: 0x000000000000 0x000000000000
0x555555559030: 0x000000000000 0x000000000000
0x555555559040: 0x000000000000 0x000000000000
0x555555559050: 0x000000000000 0x000000000000
0x555555559060: 0x000000000000 0x000000000000
0x555555559070: 0x000000000000 0x000000000000
0x555555559080: 0x000000000000 0x000000000000
0x555555559090: 0x5555555592a0 0x000000000000
0x5555555590a0: 0x000000000000 0x000000000000
```

Qu'est-ce que la HEAP

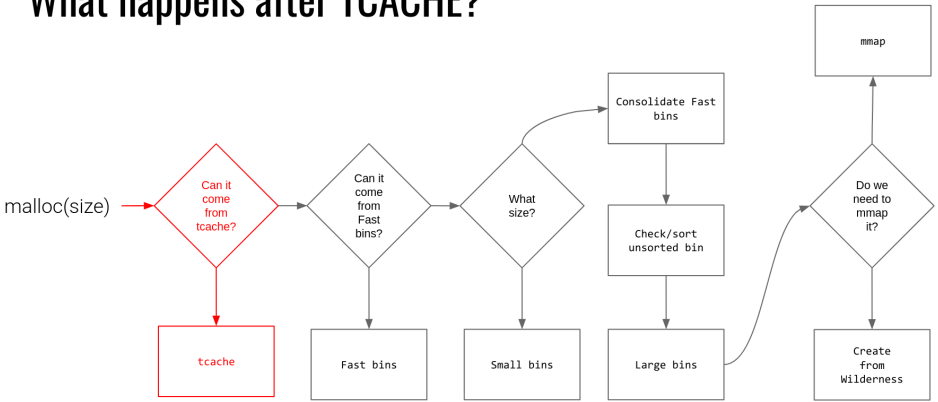
bins

- Le tcache a ses limites
  - 7 chunks free par liste
  - 64 listes (up to 1032)
- il existe d'autres types de listes (bins)
  - fastbins (liste simple, très petits chunks)
  - unsorted bin (liste double, chunks de toute taille)
  - smallbins (liste double, petits chunks)
  - largebins (liste double, gros chunks)

What happens after TCACHE?



What happens after TCACHE?



Qu'est-ce que la HEAP

main arena

- Toutes ces structures et ces listes, sont référencées et gérées par une structure qui s'appelle la main arena
- Il y a une arena par thread, celle du thread principale est la main arena.

Qu'est-ce que la HEAP

main arena

```c
void* thread_func(void* arg) {
  printf("Thread 2 starting\n");
  void * tab_alloc_t2[5];
  for (int i = 0; i < 5; i++) {
    tab_alloc_t2[i] = malloc(0x800);}
  free(tab_alloc_t2[0]);
  return NULL;
}
int main() {
  pthread_t t;
  printf("Main thread starting\n");
  pthread_create(&t, NULL, thread_func, NULL);
  for (int i = 0; i < 5; i++) {
    void* p = malloc(0x800);}
  pthread_join(t, NULL);
  return 0;
}
```

```
(gdb) info proc map
Start Addr          End Addr
0x0555555554000 0x0555555559000 [code]
0x0555555559000 0x055555557a000 [heap]
0x07ffff0000000 0x07ffff0021000 [heap2]
0x07ffff0021000 0x07ffff4000000
0x07ffff759d000 0x07ffff759e000
0x07ffff759e000 0x07ffff7d9e000
0x07ffff7d9e000 0x07ffff7da1000
0x07ffff7da1000 0x07ffff7f8a000  [libc]
0x07ffff7f8a000 0x07ffff7f97000
0x07ffff7fbd000 0x07ffff7fbf000
0x07ffff7fbf000 0x07ffff7fc3000
0x07ffff7fc3000 0x07ffff7fc5000
0x07ffff7fc5000 0x07ffff7fc7000
0x07ffff7fc7000 0x07ffff7ffe000 [ld]
0x07ffff7ffe000 0x07ffff7fff000
0x07fffffdd000 0x07ffffffff000 [stack]
```

Qu'est-ce que la HEAP

main arena

```
(gdb) ptype main_arena
type = struct malloc_state {
    __libc_lock_t mutex;
    int flags;
    int have_fastchunks;
    mfastbinptr fastbinsY[10];
    mchunkptr top;
    mchunkptr last_remainder;
    mchunkptr bins[254];
    unsigned int binmap[4];
    struct malloc_state *next;
    struct malloc_state *next_free;
    size_t attached_threads;
    size_t system_mem;
    size_t max_system_mem;
}
```

```
heap principale :
0x5555555597b0: 0x000000000000 0x000000000000
0x5555555597c0: 0x000000000000 0x000000000811
0x5555555597d0: 0x000000000000 0x000000000000

(gdb) p main_arena
$1 = {mutex = 0, flags = 0,
have_fastchunks = 0, fastbinsY = {0x0, 0x0,
0x0, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0},
top = 0x55555555c010, last_remainder = 0x0,
bins = { 0x7ffff7f88b20 <main_arena+96>, ...},
binmap = {0, 0, 0, 0}, next = 0x7ffff0000030,
next_free = 0x0, attached_threads = 1,
system_mem = 135168, max_system_mem = 135168}
```

Qu'est-ce que la HEAP

AMOSSYS

main arena

```
(gdb) ptype main_arena
type = struct malloc_state {
    __libc_lock_t mutex;
    int flags;
    int have_fastchunks;
    mfastbinptr fastbinsY[10];
    mchunkptr top;
    mchunkptr last_remainder;
    mchunkptr bins[254];
    unsigned int binmap[4];
    struct malloc_state *next;
    struct malloc_state *next_free;
    size_t attached_threads;
    size_t system_mem;
    size_t max_system_mem;
}
```

```
heap thread 2 :
0x7ffff0001b70: 0x000000000000 0x000000000000
0x7ffff0001b80: 0x000000000000 0x000000000815
0x7ffff0001b90: 0x000000000000 0x000000000000
```

```
(gdb) p *(struct malloc_state *)0x7ffff0000030
$5 = {mutex = 0, flags = 2,
have_fastchunks = 0, fastbinsY = {0x0, 0x0,
0x0, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0},
top = 0x7ffff00033b0, last_remainder = 0x0,
bins = { 0x7ffff0000b60, ... },
binmap = {0, 0, 0, 0}, next = 0x7ffff7f88ac0,
next_free = 0x0, attached_threads = 1,
system_mem = 135168, max_system_mem = 135168}
```

Pourquoi c'est 0x815 et pas 0x811?

Les Problèmes

les problèmes

Les problèmes se passent en trois temps

- Différents problèmes dans le code
- Qui créent une vulnérabilité de heap
- Et mille et une façon de les exploiter, généralement en corrompant les algos de heap

Les Problèmes

les problèmes

Les problèmes se passent en trois temps

- Différents problèmes dans le code
  - mauvais contrôle de taille de lecture/écriture
  - mauvaise gestion des pointeurs
  - résidus, pointeurs free mais toujours accessibles
- Qui créent une vulnérabilité de heap
- Et mille et une façon de les exploiter, généralement en corrompant les algos de heap

Les Problèmes

les problèmes

Les problèmes se passent en trois temps

- Différents problèmes dans le code
- Qui créent une vulnérabilité de heap
  - heap buffer overflow
  - double free
  - use after free
- Et mille et une façon de les exploiter, généralement en corrompant les algos de heap

Les Problèmes

les problèmes

Les problèmes se passent en trois temps

- Différents problèmes dans le code
- Qui créent une vulnérabilité de heap
- Et mille et une façon de les exploiter, généralement en corrompant les algos de heap
  - tcache poisonning
  - House of XXXX
  - leak libc
  - ....

Les Problèmes

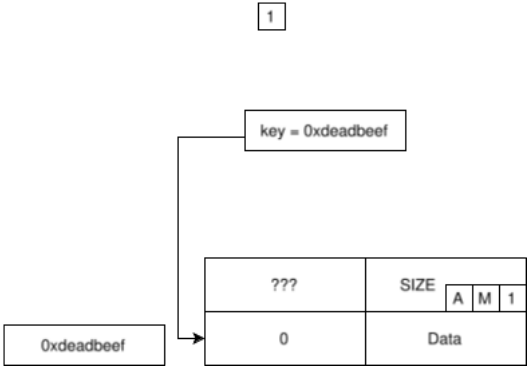
Double Free 1

→ free?

Les Problèmes

Double Free 1

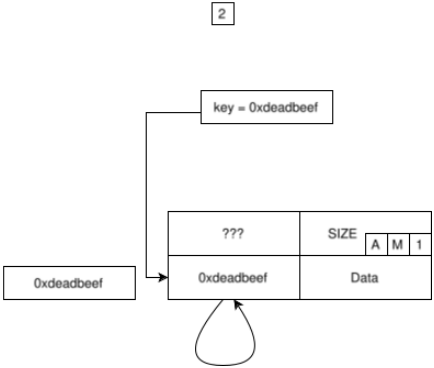
- free(0xdeadbeef);
- refree?



Les Problèmes

Double Free 1

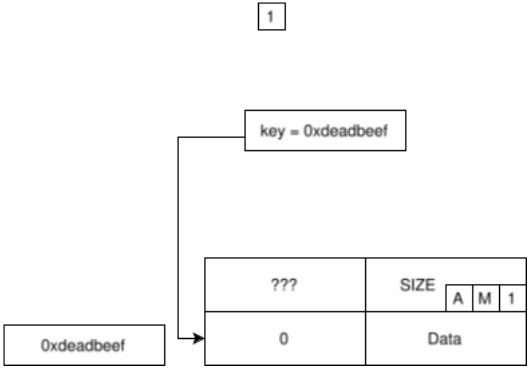
- free(0xdeadbeef);
- free(0xdeadbeef);
- alloc?



Les Problèmes

Double Free 1

- free(0xdeadbeef);
- free(0xdeadbeef);
- void * a = malloc(16);
- alloc?



Les Problèmes

Double Free 1

- free(0xdeadbeef);
- free(0xdeadbeef);
- void * a = malloc(16);
- void * b = malloc(16);

Problème ?

Les Problèmes

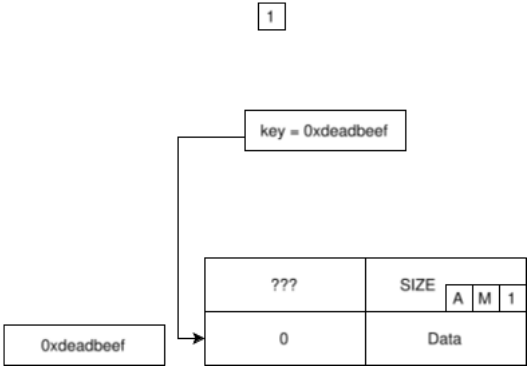
Double Free 2

→ free?

Les Problèmes

Double Free 2

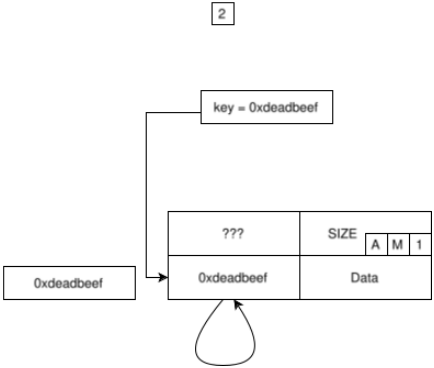
- free(0xdeadbeef);
- free?



Les Problèmes

Double Free 2

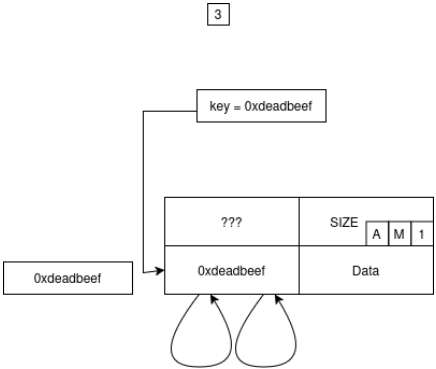
- free(0xdeadbeef);
- free(0xdeadbeef);
- free?



Les Problèmes

Double Free 2

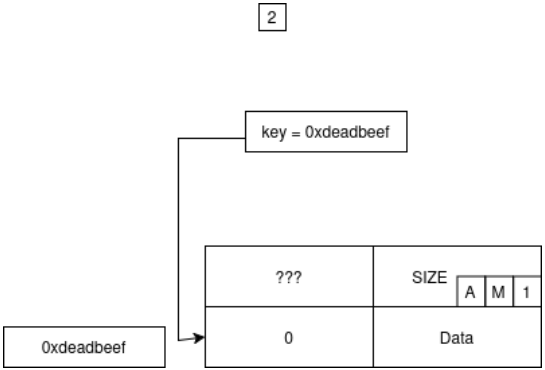
- free(0xdeadbeef);
- free(0xdeadbeef);
- free(0xdeadbeef);
- alloc?



Les Problèmes

Double Free 2

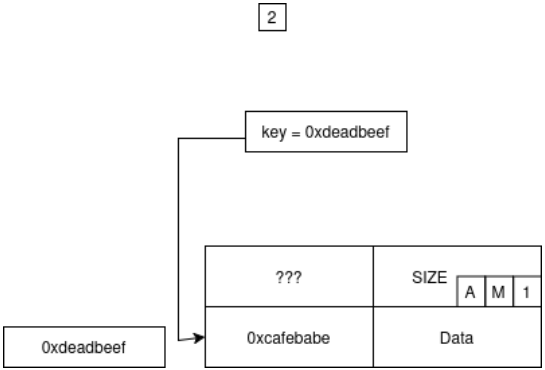
- free(0xdeadbeef);
- free(0xdeadbeef);
- free(0xdeadbeef);
- void * a = malloc(16);
- modif?



Les Problèmes

Double Free 2

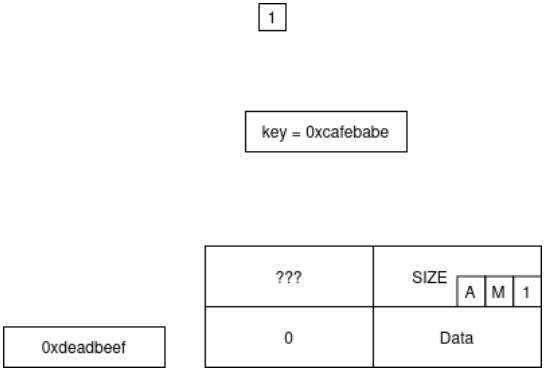
- free(0xdeadbeef);
- free(0xdeadbeef);
- free(0xdeadbeef);
- void * a = malloc(16);
- `strncpy(a, "\xfe\xaf\xdc\xba", 4);`
- realloc?



Les Problèmes

Double Free 2

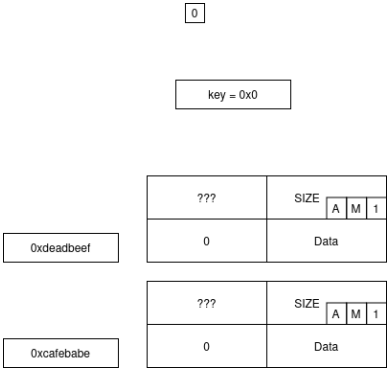
- free(0xdeadbeef);
- free(0xdeadbeef);
- free(0xdeadbeef);
- void * a = malloc(16);
- `strncpy(a, "\xfe\xaf\xdc\xba", 4);`
- void * b = malloc(16);
- realloc?



Les Problèmes

Double Free 2

- free(0xdeadbeef);
- free(0xdeadbeef);
- free(0xdeadbeef);
- void * a = malloc(16);
- `strncpy(a, "\xfe\xaf\xdc\xba", 4);`
- void * b = malloc(16);
- void * c = malloc(16);

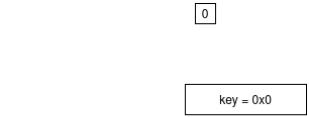


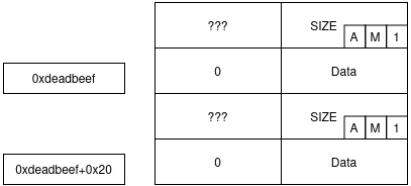
Pourquoi était-ce necessaire de le free trois fois ?

Les Problèmes

Use After Free

- free?

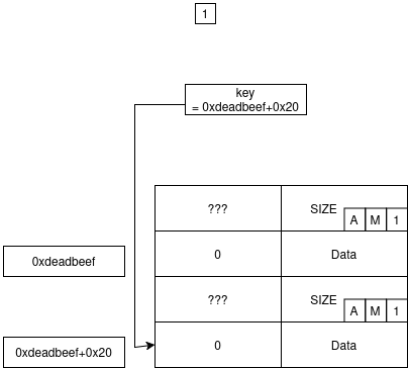




Les Problèmes

Use After Free

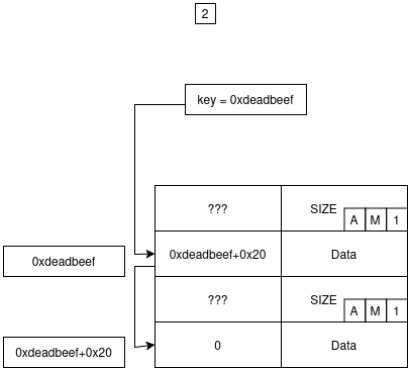
- free(0xdeadbeef + 0x20);
- free?



Les Problèmes

Use After Free

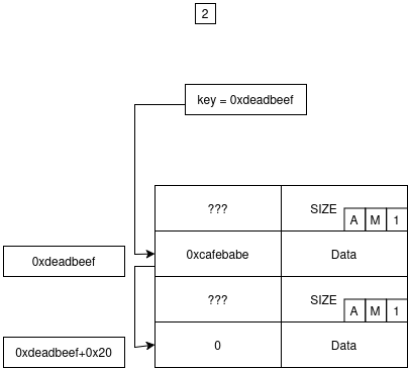
- free(0xdeadbeef + 0x20);
- free(0xdeadbeef);
- read / write ?



Les Problèmes

Use After Free

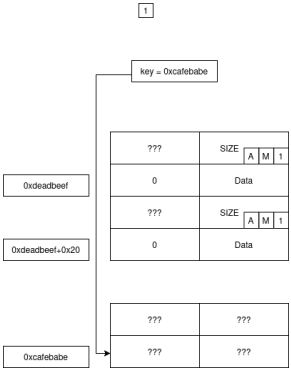
- free(0xdeadbeef + 0x20);
- free(0xdeadbeef);
- read / write;
- alloc?



Les Problèmes

Use After Free

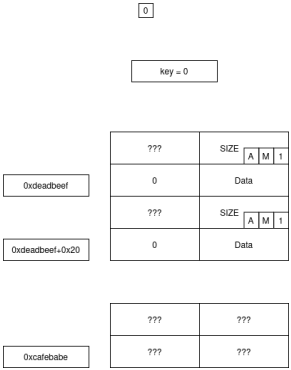
- free(0xdeadbeef + 0x20);
- free(0xdeadbeef);
- read / write;
- void * a = malloc(16);
- realloc?



Les Problèmes

Use After Free

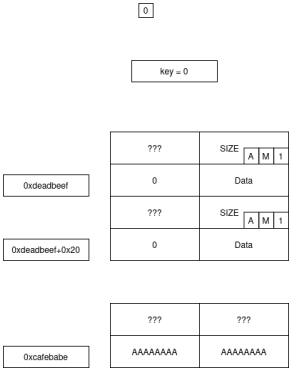
- free(0xdeadbeef + 0x20);
- free(0xdeadbeef);
- read / write;
- void * a = malloc(16);
- void * b = malloc(16);
- after?



Les Problèmes

Use After Free

- free(0xdeadbeef + 0x20);
- free(0xdeadbeef);
- read / write;
- void * a = malloc(16);
- void * b = malloc(16);
- leak or write whatever we want wherever we want



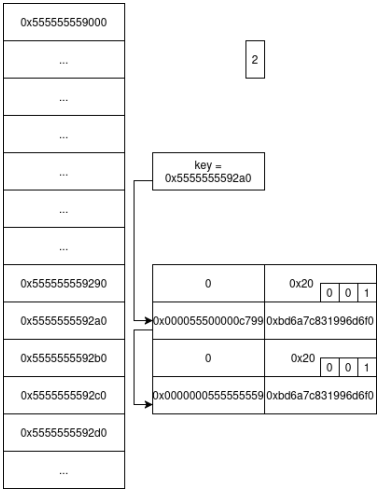
Protections

Des protections

- Les developpeurs de la libc savent que les programmeurs font des erreurs
- Et donc il ya plusieurs protections
- À la fois pour éviter qu'un programme fasse n'importe quoi
- Mais aussi pour rendre l'exploitation plus complexe

Protections

Safe Linking



- Quand un chunk est free, est mis dans chunk[2],
- non pas le pointeur vers le chunk suivant,
- mais un pointeur manglé.
- `chunk[2] = (chunk[2] >> 12 ) ^ ptr`
- sur l'exemple :
- `key -> 0x5555555592a0 = 0x55500000c799`
- `(0x5555555592a0 >> 12) ^ 0x5555555592c0`
- `= 0x555555559 ^ 0x5555555592c0 = 0x55500000c799`

Safe Linking

De quoi est-ce que ça protège ?

Safe Linking

De quoi est-ce que ça protège ?

- Ça n'empèche pas la corruption de la liste du tcache.
- Mais ça rend plus 'difficile' une BONNE corruption.
- Il nous faut une information en plus afin de faire une bonne corruption, laquelle ?

Protections

Safe Linking

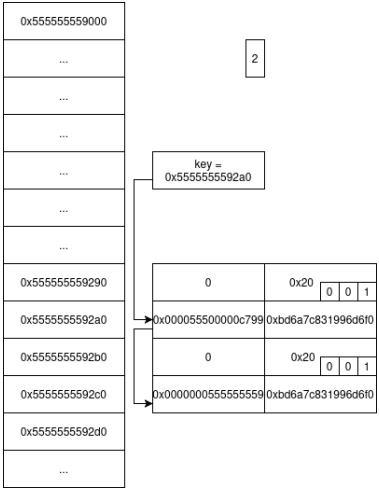
De quoi est-ce que ça protège ?

- Ça n'empèche pas la corruption de la liste du tcache.
- Mais ça rend plus 'difficile' une BONNE corruption.
- Il nous faut une information en plus afin de faire une bonne corruption, laquelle ?

Il faut maintenant connaître une adresse de heap afin de connaître le mangling.

Protections

Double Free Key



- Quand un chunk est free, est mis dans chunk[3],
- Une key random, mais commune à tous les chunks.
- Un peu comme le canary, même si il est random, une fois connu, il est connu pour toute la durée de vie du programme.
- Ainsi quand un chunk est free,
  - la libc verifie que chunk[3] != double free key
  - si c'est égal, ça detecte un double free
  - sinon ça free le chunk.

Double Free Key

De quoi est-ce que ça protège ?

Protections

Double Free Key

De quoi est-ce que ça protège ?

- Ça n'empèche pas un double free.
- Mais ça rend plus 'difficile' de le faire.
- Il nous faut une primitive en plus afin de faire une bonne corruption, laquelle ?

Protections

Double Free Key

De quoi est-ce que ça protège ?

- Ça n'empèche pas un double free.
- Mais ça rend plus 'difficile' de le faire.
- Il nous faut une primitive en plus afin de faire une bonne corruption, laquelle ?

Il faut avoir la possibilité d'écrire (UAF/W) ou overflow sur cet endroit.